

E-Commerce Sales Analysis Using SQL

1. Top 10 Customers By Revenue

Objective: Identify the highest revenue-generating customers.

```
SELECT customer_id,  
       SUM(quantity * unit_price) AS revenue  
FROM e_data  
GROUP BY customer_id  
ORDER BY revenue DESC  
LIMIT 10;
```

2. Revenue Contribution By Category

Objective: Calculate category contribution to total revenue.

```
WITH category_revenue AS (  
    SELECT category,  
           SUM(quantity * unit_price) AS revenue  
    FROM e_data  
    GROUP BY category  
)  
total_revenue AS (  
    SELECT SUM(revenue) AS total_revenue  
    FROM category_revenue  
)  
SELECT cr.category,  
       cr.revenue,  
       ROUND((cr.revenue * 100.0 / tr.total_revenue),2) AS revenue_contribution  
FROM category_revenue cr  
CROSS JOIN total_revenue tr  
ORDER BY revenue_contribution DESC;
```

3. Find Repeat Customers

Objective: Find customers with more than one order.

```
SELECT customer_id,  
       COUNT(DISTINCT order_id) AS total_orders  
FROM e_data  
GROUP BY customer_id  
HAVING COUNT(DISTINCT order_id) > 1;
```

4. Most Returned Products

Objective: Identify products with highest returns.

```
SELECT product_name,  
       COUNT(*) AS return_count  
FROM e_data  
WHERE order_status = 'Returned'  
GROUP BY product_name  
ORDER BY return_count DESC;
```

5. Monthly Revenue Trend

Objective: Analyze revenue trend over time.

```
SELECT YEAR(order_date) AS year,  
       MONTH(order_date) AS month,  
       SUM(unit_price * quantity) AS total_revenue  
FROM e_data  
GROUP BY year, month  
ORDER BY year, month;
```

6. Running Total Revenue

Objective: Track cumulative revenue.

```
SELECT order_date,
       SUM(unit_price * quantity) AS revenue,
       SUM(SUM(unit_price * quantity))
       OVER (ORDER BY order_date) AS running_total
FROM e_data
GROUP BY order_date;
```

7. Rank Customers By Spending

Objective: Rank customers by spending.

```
WITH customer_spending AS (
  SELECT customer_id,
         SUM(unit_price * quantity) AS total_spending
  FROM e_data
  GROUP BY customer_id
)
SELECT customer_id,
       total_spending,
       DENSE_RANK() OVER(
         ORDER BY total_spending DESC
       ) AS customer_rank
FROM customer_spending;
```

8. Top Product In Each Category

Objective: Find best product per category.

```
SELECT *
FROM (
  SELECT category,
         product_name,
         SUM(unit_price * quantity) AS total_revenue,
         RANK() OVER(
           PARTITION BY category
           ORDER BY SUM(unit_price * quantity) DESC
         ) AS rnk
  FROM e_data
  GROUP BY category, product_name
) x
WHERE rnk = 1;
```

9. Customers Spending Above Average

Objective: Find premium customers.

```
SELECT customer_id,
       SUM(unit_price * quantity) AS total_spending
FROM e_data
GROUP BY customer_id
HAVING SUM(unit_price * quantity) >
(
  SELECT AVG(total_spending)
  FROM (
    SELECT customer_id,
           SUM(unit_price * quantity) AS total_spending
    FROM e_data
    GROUP BY customer_id
  ) x
);
```

10. Inactive Customers

Objective: Customers inactive for 90 days.

```
SELECT customer_id,
```

```

        MAX(order_date) AS last_order_date
FROM e_data
GROUP BY customer_id
HAVING MAX(order_date) <
DATE_SUB(
    (SELECT MAX(order_date) FROM e_data),
    INTERVAL 90 DAY
);

```

11. Customers Buying Multiple Categories

Objective: Cross-category buyers.

```

SELECT customer_id,
       COUNT(DISTINCT category) AS total_categories
FROM e_data
GROUP BY customer_id
HAVING COUNT(DISTINCT category) > 1;

```

12. Revenue Lost Due To Returns

Objective: Measure return losses.

```

SELECT SUM(quantity * unit_price) AS revenue_lost
FROM e_data
WHERE order_status = 'Returned';

```

13. Most Popular Payment Method Per Country

Objective: Preferred payment method by country.

```

SELECT *
FROM (
    SELECT country,
           payment_method,
           COUNT(*) AS total_orders,
           RANK() OVER(
               PARTITION BY country
               ORDER BY COUNT(*) DESC
           ) AS rnk
    FROM e_data
    GROUP BY country, payment_method
) x
WHERE rnk = 1;

```

14. Month-over-Month Growth Rate

Objective: Measure monthly growth.

```

WITH monthly_revenue AS (
    SELECT YEAR(order_date) AS year,
           MONTH(order_date) AS month,
           SUM(quantity * unit_price) AS revenue
    FROM e_data
    GROUP BY year, month
)
SELECT month,
       year,
       revenue,
       LAG(revenue) OVER(ORDER BY year, month) AS previous_month_revenue,
       ((revenue - LAG(revenue) OVER(ORDER BY year, month))
        * 100.0 / LAG(revenue) OVER(ORDER BY year, month)) AS growth_rate
FROM monthly_revenue;

```

15. Products Never Returned

Objective: Products with zero returns.

```

SELECT DISTINCT e1.product_name
FROM e_data e1
WHERE NOT EXISTS (

```

```

SELECT 1
FROM e_data e2
WHERE e2.product_name = e1.product_name
      AND e2.order_status = 'Returned'
);

```

16. Products Contributing More Than 10% Of Category Revenue

Objective: Key category drivers.

```

WITH product_sales AS (
  SELECT category,
         product_name,
         SUM(unit_price * quantity) AS product_revenue
  FROM e_data
  GROUP BY category, product_name
)
SELECT *
FROM product_sales p
WHERE product_revenue >
(
  SELECT SUM(unit_price * quantity) * 0.10
  FROM e_data e
  WHERE e.category = p.category
);

```

17. Products With Declining Monthly Sales

Objective: Detect declining products.

```

WITH monthly_sales AS (
  SELECT product_name,
         YEAR(order_date) AS y,
         MONTH(order_date) AS m,
         SUM(unit_price * quantity) AS sales
  FROM e_data
  GROUP BY product_name, y, m
)
SELECT product_name
FROM (
  SELECT *,
         LAG(sales) OVER(
           PARTITION BY product_name
           ORDER BY y, m
         ) AS previous_sales
  FROM monthly_sales
) x
WHERE sales < previous_sales;

```

18. Fastest Growing Product Month-over-Month

Objective: Find highest absolute growth.

```

WITH monthly_sales AS (
  SELECT product_name,
         YEAR(order_date) AS y,
         MONTH(order_date) AS m,
         SUM(unit_price * quantity) AS sales
  FROM e_data
  GROUP BY product_name, y, m
)
SELECT product_name,
       y, m,
       sales,
       previous_sales,
       ROUND((sales - previous_sales),2) AS growth
FROM (
  SELECT *,
         LAG(sales) OVER(
           PARTITION BY product_name
           ORDER BY y, m
         ) AS previous_sales
  FROM monthly_sales
)

```

```
) x  
WHERE previous_sales IS NOT NULL  
ORDER BY growth DESC  
LIMIT 1;
```